

Data Augmentation for Software Vulnerability Assessment Predicted Model Final Reports

Dinh Tuan Lu
a1866436

August 18, 2023

Report submitted for **Data Science Research Project Part B** at
the School of Mathematical Sciences, University of Adelaide



THE UNIVERSITY
of ADELAIDE

Project Area: **Data Augmentation techniques effect on NLP
Prediction Model**

Project Supervisor: **Dr. Triet Huynh Minh Le**

In submitting this work I am indicating that I have read the University's Academic Integrity Policy. I declare that all material in this assessment is my own work except where there is clear acknowledgement and reference to the work of others.

I give permission for this work to be reproduced and submitted to other academic staff for educational purposes.

OPTIONAL: I give permission this work to be reproduced and provided to future students as an exemplar report.

Abstract

Software and technologies have become indispensable tools for individuals and businesses, facilitating most of our daily activities. Eliminating vulnerabilities in software that can expose systems to potential security breaches will prevent substantial losses for businesses and users. The National Vulnerability Database (NVD) offers public access to the Common Vulnerabilities and Exposures (CVE) data set, enabling developers to identify and follow up with the latest vulnerabilities. By leveraging machine learning and deep learning, particularly Natural Language Processing (NLP), in combination with NVD data to create models that assess the impact of risk descriptions. These models are hindered by challenges stemming from data imbalances during classification model construction.

To mitigate these challenges, we explore data augmentation techniques, a prevalent approach in machine learning that generates new data samples from existing data. In the context of NLP, data augmentation enhances the performance of machine learning models by introducing diverse variations and generalizing the model. This prevents issues like over-fitting, bias, and potential loss of meaning within vulnerability descriptions.

This report uses the NVD dataset to compare the performance of machine learning and deep learning models. It investigates data augmentation techniques to assess their impact on model performance. The data augmentation techniques are more effective on deep learning models.

1 Introduction

Identifying and managing software vulnerabilities are critical tasks within the realm of cyber-security. Addressing software vulnerabilities early in the development process is essential to ensure software systems' security and integrity, protect user data, and maintain the trust of both businesses and individuals.

The National Vulnerability Database (NVD) plays a pivotal role in increasing the quality of software securities. The NVD is a comprehensive repository maintained by the National Institute of Standards and Technology (NIST) that catalogues and provides detailed information about known software vulnerabilities. This database is a central resource that gathers data from various sources, including security researchers, vendors, and organizations, to create a standardized format for sharing information about vulnerabilities.

In this project, we use the NVD database to train both machine learning (ML) and deep learning (DL) models. Data augmentation techniques such as data sampling techniques like over-sampling and under-sampling. Some other data augmentation techniques for NLP, such as random insertion, random substitution and random deletion. We also use data augmentation using synonyms to experiment if it boosts the performance of the prediction models.

2 Background

2.1 Prediction models for vulnerabilities and its challenges

2.1.1 Exploratory data analysis of vulnerabilities

Vulnerabilities in software engineer is a critical problem that is focus by many parties. Department of Homeland Security in the United States and their US-CERT (United States Computer Emergency Readiness Team) had funded professional developer to create the NVD database ¹. It is a standardized assessment metric of security vulnerabilities, which frequently updated since its initial introduction in 2005.[18]. We use the Common Vulnerability Scoring System (CVSS) to get the vulnerability characteristic. This metric is used for scoring the severity of computer system security vulnerabilities.

For this project, we use the NVD database and the CVSS to build the prediction model. For training and testing, we use the vulnerability description and its metrics. Using APIs <https://nvd.nist.gov/developers/vulnerabilities> on February 2023, including 180087 samples downloaded. In this project, we use CVSS v2 (CVSS v3.1 is the latest) metrics for measuring the vulnerability character for modern technologies and platforms. The prediction model build upon NVD aims to evaluate the security vulnerability impacts such as complexity, integrity, and availability can have on software based on its description.

After Exploratory Data Analysis (EDA), the metrics score from various outputs in the NVD database has an imbalanced distribution between each output class, shown in the table 1. The two most imbalanced output targets are `cvss2_access_vector` and `cvss2_authentication`.

2.1.2 Prediction models Vulnerability assessment tasks

The Australian Government's Cooperative Research Centres Program has funded a survey for data-driven Software Vulnerability Assessment and Prioritization [12]. Finding the best practices, limitations, and potential solutions to address the Software Vulnerability Assessment is crucial in the software industry. The vulnerability impact assessment specification requires manual analysis tasks from security experts in software development. These analysis and evaluation steps required a long time to ensure the accuracy of the impact. This is affecting the response time to urgent software security risks. We can accelerate the process by building machine and deep learning prediction models. The data-set input is the

¹<https://www.cisa.gov/>

#	Vulnerability metric	Class 1	Class 2	Class 3
chunk_1	cvss2_access_vector	30726	5215	76
	cvss2_access_complexity	25611	1833	8573
	cvss2_authentication	34630	1380	7
	cvss2_confidentiality_impact	6585	18699	10733
	cvss2_integrity_impact	6343	20515	9159
	cvss2_availability_impact	7450	18115	10452
	cvss2_base_score	16690	16949	2378
chunk_2	cvss2_access_vector	60956	9074	2004
	cvss2_access_complexity	42875	3106	26053
	cvss2_authentication	66153	5823	58
	cvss2_confidentiality_impact	15126	34122	22786
	cvss2_integrity_impact	14581	38772	18681
	cvss2_availability_impact	17388	31894	22752
	cvss2_base_score	29990	36806	5238
chunk_3	cvss2_access_vector	91727	13819	2505
	cvss2_access_complexity	62014	4212	41825
	cvss2_authentication	96274	11702	75
	cvss2_confidentiality_impact	22243	51086	34722
	cvss2_integrity_impact	21545	55896	30610
	cvss2_availability_impact	25892	46131	36028
	cvss2_base_score	41103	58175	8773

Table 1: Number of samples in each class for individual output on the training set

vulnerability description, so we can use the NLP pre-processing to train and test the performance of the models. Most of the descriptions from NVD are from the data collection process on incidents that developers around the world reported. Creating new knowledge to help balance the data is challenging and requires strict standards based on security experts' analysis.

2.1.3 Data sampling for NLP problem

Under-sampling entails equalizing class samples by removing instances from the majority class, aligning it with the minority class's composition [26]. Random under-sampling (RUS) randomly eliminates samples without replacement. However, this method could stop crucial selections and can reduce model accuracy [26]. In contrast, Oversampling increases instances in the minority class, which often yields robust outcomes [14]. ROS still only uses the minority class data, resulting in the absence of

new information generation.

2.1.4 Data augmentation for NLP problem

Data augmentation (DA) is a method used to boost the data diversity based on existing data[20]. It is an essential step when data collecting is difficult. The goal is creating modified copies of current data or generating new information with the same context as the sample. Data augmentation is commonly used in Computer Vision (CV) models as a regularizer which prevent the model from over-fitting. However, no standard framework exists for DA techniques in the NLP space. Inspired by computer vision (CV) data augmentation techniques, we can implement some methods such as sentence augmentation techniques like random insertion, random deletion and random substitution of words. Others methods are creating new words by adding synonyms and doing back-translation.

2.1.5 Related works: Vulnerability assessment prediction model

Many machine learning and deep learning applications exist in the Nature Language Processing (NLP) space. Some examples that are well-received by the public are chatGPT, Krisp, and Podcastle. With the rapid development and reliability of these models, multiple machine learning and deep learning-based vulnerability detection models have already been built. Let's look at some of these projects.

One of them is conducted by Spanos et al. (2018) [21] about prediction models for software vulnerability characteristics and severity. Multiple prediction models like Random Forest, XGBoost, and Decision Tree are experimented. The results show that all algorithms have moderate to good performance. For access vector and access complexity, however, the prediction results are mostly wrong. The unbalanced data-set distribution problem is suggested to be one of the key problems and suggested boosting the minority classes by different algorithms in Tsoumakas et al. (2011) literature [21].

Costa et al.(2022)[4] works with DistilBERT, which is a natural language pre-processing technique. From all the experiments, the pre-processing techniques perform well in binary classes, particularly on imbalanced data sets. This shows that there is research to improve the vulnerability assessment prediction model by improving the text data pre-processing techniques.

Scandariato and colleagues (2021)[23] employ text mining techniques, utilizing the frequency of particular terms, to detect vulnerabilities. Similarly, Ziems, Noah, and Wu (2021)[27] explore security vulnerabilities by

analyzing C code as input to proactively identify potential risks. These investigations indicate the promise of utilizing deep learning models for the classification of vulnerabilities.

2.1.6 Related works: Data augmentation for NLP models

Due to the limitation of the vocabulary of professional programming, it is hard to generate new data for the project with data collection and traditional methods. Data augmentation is one of the solutions that is commonly used in machine learning and deep learning. Multiple data augmentation implementations and ideas for the textual analysis model are experimented with in Bayer et al. (2021) [1]. It provided insights into the need to improve the performance of unbalanced data sets. Many challenges, such as minimal benefits for pre-trained models such as BERT and RoBERTa, domain-specific vocabulary. This shows the advantage of implementing data augmentation in the project. This is relevant in our cases so we can use their framework and approaches, which had a proper build pipeline to operate.

GPT is the new popular AI tool for performing tasks using the neural network-trained generalized AI processing model. Some application utilizes the GPT version 2 models with engineered prompts, which improves all the classification reinforcement learning models. These methods are called Data boosts. Proposed by Ruibo et al. (2020) [15], It is a condition text augmentation method. It is good to notice that heavy investment in computing resources to maintain the context of the sample using multiple pre-built language models is required by this method.

2.2 Previous results in project part A

In the previous part of the project, We built the foundation pipeline for the machine learning-based vulnerabilities in part A of the project. Data augmentation techniques, such as random oversampling on all minority classes, have been tested on the machine learning model. Overall, the results show the model performs best without performing any data augmentation. On closer inspection, the random oversampling and random data insertion augmentation techniques have a higher score in some evaluation techniques and models. Ensemble models such as RF, XGBoost and LGBM perform well with random oversampling technique which is slightly outperforming the other methods. Data augmentation such as under-sampling, random substitution augmentation, and random deletion augmentation have a lower result of less than 0.3 MCC score. The results are expected due to new samples providing less information and

reducing variety. The context of vulnerability description is not prioritized in the process for these data augmentation techniques.

3 Methods

3.1 Project coding framework

This project builds prediction models for multi-classifying vulnerability characteristics in software engineering. The main focus of these models is tackling data distribution imbalanced issues using data augmentation. We built both Machine learning and Deep learning models for more variety in the experiment to understand the effect of data augmentation. Vulnerability impact prediction models are tuned and combine data sampling and multiple data augmentation techniques. The results of the data set without data augmentation can be used as a main score to compare and evaluate the effectiveness of data augmentation techniques ranging from simple to complex. The framework of the project can be put into a pipeline. There are five main steps in the framework: Obtaining and pre-processing data, balancing the class distribution for each chunk of the data set, feature extraction to prepare for the model training, fitting and tuning each model with its hyper-parameter, and evaluation using all the metrics. The detail of the framework is in the following Figure 1:

3.2 Data-set obtain and stored

The training and testing data set is derived from the NVD database description and CVSS v2 characteristic. For experimentation, we use the following vulnerability impact metrics, which we can call from the database: [cve id, published time, impact metrics, base score, and other technical descriptions of the vulnerabilities]. Table 2 lists six vulnerability characteristics selected for the model fitting and predicting. For this project, we use the data from the API, which contains 180087 samples of vulnerability from 1988 until 2022. All of them are stored in the local database pre-processed. The data is divided into 5 equal chunks for training and testing for a more robust analysis. Each iteration of model evaluation requires a minimum of 1 chunk of training, 1 chunk for validation and 1 chunk for testing. After each iteration, the training set size increases by 1 chunk, so the training set size equals 3 chunks in the final run. After running, the optimal result is the maximum of all three iterations to see the best-performing models after tuning. These ensure we can capture the best performance for each model and data augmentation techniques.

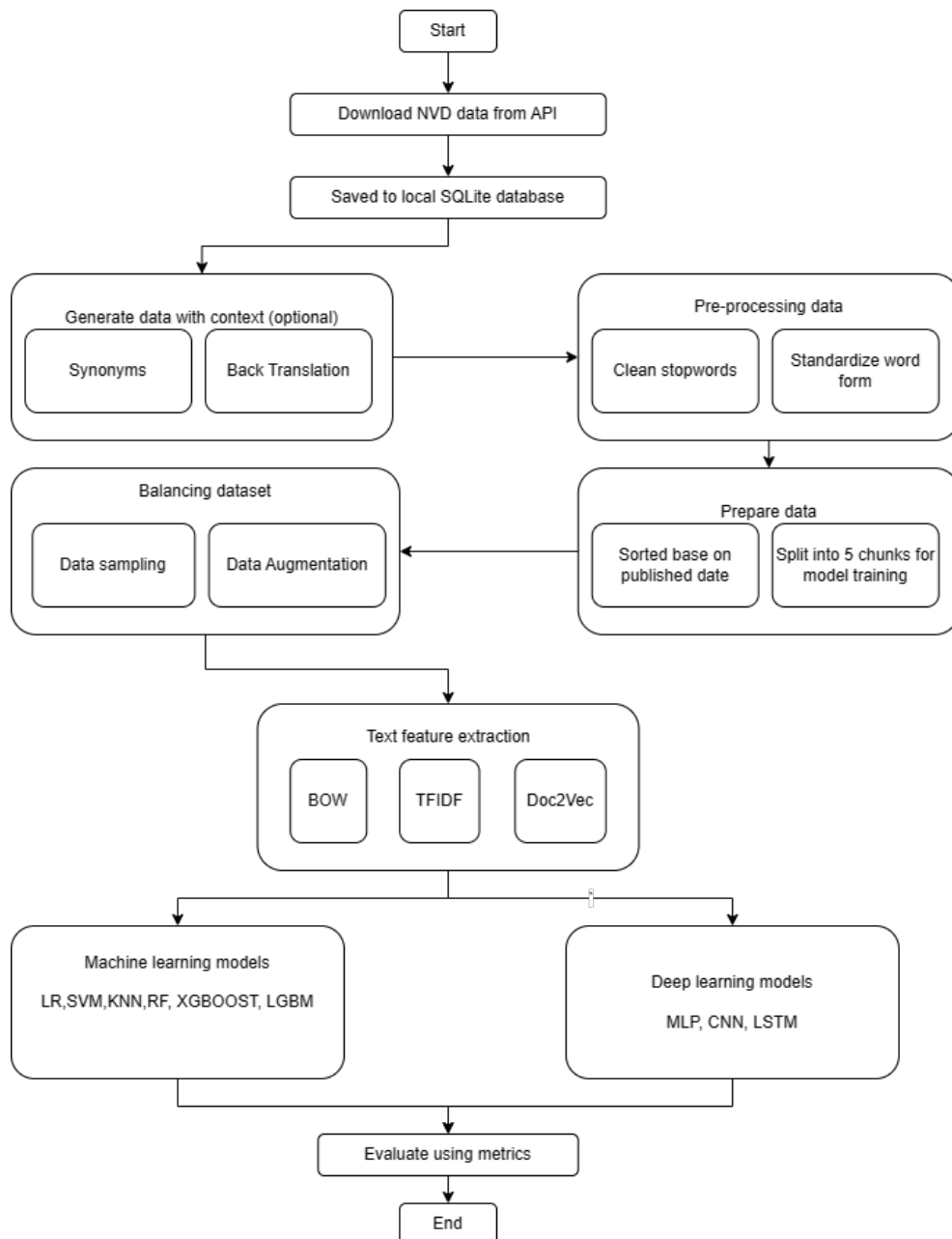


Figure 1: Pipeline of the project

Table 2: Vulnerability characteristics description and values[21]

Name	Description	Values
Access Vector	The way with which a vulnerability can be exploited.	Local (L), Adjacent Network (AN), Network (N)
Access Complexity	The level of difficulty to access the candidate target.	High (H), Medium (M), Low (L)
Authentication	The level of authentication needed to access the vulnerable system.	Multiple Instances (MI), Single Instance (SI), None (N)
Confidentiality Impact	The impact on the confidentiality of the affected system	None (N), Partial (P), Complete (C)
Integrity Impact	The impact on the integrity of the affected system.	None (N), Partial (P), Complete (C)
Availability Impact	The impact on the availability of the affected system.	None (N), Partial (P), Complete (C)

3.3 Pre-processing data

The local SQLite database has the following schema for pre-processing data, modelling, and evaluation :[cve_id, desc, cvss2_vector_string, cvss2_access_vector, cvss2_access_complexity, cvss2_authentication, cvss2_confidentiality_impact, cvss2_integrity_impact, cvss2_availability_impact, cvss2_base_score, 'published_date']. We want to build a prediction model for future vulnerability assessment so the data is sorted based on the published_date for training and testing of the model. The description for the database needed to be pre-processed and transformed for NLP modelling. This is a standard step of NLP to convert all sentences into a structured feature framework that could be used for prediction model training. Eliminating stop-words, which are the most common words in the English language (like articles, prepositions, pronouns, conjunctions) [8] is one of the essential steps to remove all of the low-level words from the description. These word does not support the model training. By trimming down these data, the computation resource can be reduced without any negative reduction in performance on the model. It also reduces training time and memory used by the

computer.

In table 1, `cvss2_access_vector` and `cvss2_authentication` have the most severe imbalance between the majority class and minority class. This can cause bias in the model, which is not optimal when predicting high-risk vulnerabilities. The class in all output is processed and converted into numbers, for example: `[0,1,2]`. The table 1 also shows that `cvss2_access_vector` and `cvss2_authentication` has the most severe imbalance while `cvss2_confidentiality_impact` and `cvss2_availability_impact` are more balanced. The experiment in this project focuses on the effect of the imbalanced data and the effectiveness of the data sampling and data augmentation. We will run the model without the balancing technique as the baseline for comparison. Data sampling and augmentation implementation are evaluated by their performance on each model and output.

3.4 Text data feature extraction

In our study, we implemented a range of techniques, including Bag of Words (BOW), TFIDF, and Doc2Vec, to carry out feature extraction from the textual data. The Bag of Words method transforms distinct words into fixed-length vectors, considering the frequency of phrase occurrences in the corpus. Meanwhile, TF-IDF assigns weights to terms based on both the Inverse Document Frequency (IDF) and Term Frequency (TF) values within the dataset. These approaches offer simplicity and adaptability, proving to be versatile for extracting text features. However, it's noteworthy that these two methods might fail to preserve contextual nuances, which could impact their performance in context-sensitive data augmentation scenarios.

A notable addition to our feature extraction arsenal is the Doc2Vec technique, an extension of the Word2Vec model that introduces a paragraph vector to enhance its capabilities[7]. Doc2Vec demonstrates robustness even when trained on extensive external corpora and benefits from pre-trained word embeddings[7]. This attribute holds particular significance considering the scale of the National Vulnerability Database (NVD) dataset and the vulnerability description samples within both the original dataset and the augmented samples generated². This incorporation of Doc2Vec further enriches our methodology, ensuring a holistic approach to feature extraction that accommodates the intricate characteristics of the data.

There is more text data feature extraction technique, but the main focus of this project is experimenting with data augmentation, so three

²<https://whites.agency/blog/introducing-doc2vec-algorithm-by-nlp-and-its-use-in-seo/>

of the most common technique does provide us with valuable insight.

3.5 Data sampling technique

In the scope of this project, two distinct data sampling strategies were employed: over-sampling and under-sampling. Data sampling techniques constitute fundamental tools in machine learning and deep learning, facilitating the creation of data subsets derived from the original data set. These techniques prove invaluable in addressing various challenges, including class imbalance and the risk of over-fitting, enhancing the overall robustness of models.

Over-sampling involves augmenting the number of instances within minority classes to rectify class distribution imbalances. In the context of the National Vulnerability Database (NVD), characterized by a multi-class output classification, this process generates additional samples for the underrepresented classes. The overarching goal of this experiment revolves around generating a substantial volume of data to meticulously explore the intricate relationship between sample size and its consequent impact on the efficacy of the vulnerability assessment model.

Conversely, under-sampling entails the random removal of samples from the majority classes. While this approach serves to address imbalance, it can notably affect the model's performance in cases of pronounced output imbalance, such as in the categories of "cvss2_authentication" or "cvss2_access_vector." The outcome of under-sampling in these instances leads to a considerable reduction in sample size, a factor that bears a significant influence on the overall model behaviour and effectiveness.

3.6 Data augmentation technique

When implementing data augmentation techniques, we've employed an approach that mirrors the principles of data sampling strategies. This process commences with determining the requisite number of samples for each class encompassing the target vulnerabilities. Subsequently, leveraging the samples from the minority classes, we generate augmented versions of the model.

Our augmentation methodology encompasses diverse tactics. To execute random insertion, we rely on the vocabulary of the training dataset. Specifically, we select the top 30% most frequent words for insertion into the newly augmented samples. For random deletion, words are randomly chosen and extracted from the samples, yielding fresh augmented data. Additionally, we combine both these techniques for random substitution.

Here, we opt for a random word, which we then replace exclusively with words within the top 30% of frequency in the data set. In a parallel avenue, we harness the power of synonyms for data augmentation. This technique operates by randomly substituting a chosen word with one of its synonyms.

To facilitate this synonym-based augmentation, we incorporate a thesaurus vocabulary. This repository of synonyms plays a pivotal role in generating new variations of the dataset by substituting words with synonymous alternatives. This comprehensive augmentation framework weaves together multiple strategies to bolster our data set's diversity and ultimately enhance the robustness of our model.

3.7 Machine learning models

We chose six diverse machine learning algorithms for the vulnerability classification experiment. These are Logistic Regression (LR), Support Vector Machine (SVM), K-Nearest Neighbours (KNN), Random Forest (RF), Boosting (XGBoost), and Light Gradient Boosting Machine (LGBM). This selection is intended to gather results from a broad spectrum of machine learning techniques, ranging from fundamental classification methods to more advanced ensemble classifiers.

Basic algorithms such as Logistic Regression (LR), Support Vector Machine (SVM), and K-Nearest Neighbours (KNN). These algorithms are widely recognized and utilized for multiple classification tasks, offering solid performance and potential for optimization. Their flexibility allows for tuning, enabling the enhancement of their predictive prowess.

Furthermore, our ensemble classifier choices include Random Forest, renowned for its efficacy in mitigating overfitting (Bentéjac et al., 2009) [3], as well as the XGBoosting algorithm, celebrated for its exceptional performance as evidenced by numerous studies (Bentéjac et al., 2009 [2], Verlade et al., 2023 [24]).

To refine the performance of all these machine learning models, we employed gridCV for hyper-parameter tuning. The specific hyper-parameters used in the tuning process are delineated in section Model hyper-parameters found in the appendices. The tuning process aims to extract optimal performance from each model, thus ensuring the most accurate and insightful vulnerability classification outcomes.

3.8 Deep learning models

In this project, we have implemented three distinct deep learning models - Multilayer Perceptron (MLP), Convolutional Neural Network (CNN),

and Long Short-Term Memory (LSTM) - for the realm of Natural Language Processing (NLP) tasks. This choice stems from their individual strengths in encapsulating different facets of textual data, resulting in a holistic approach to text analysis.

The Multilayer Perceptron (MLP) stands out as a versatile neural network architecture adept at encapsulating semantic associations within sentences. Zhang and Lee (2018) [17] provide compelling support for this approach, highlighting the MLP model's efficacy in discerning intricate patterns within textual data. Its capability to grasp nuanced relationships lends itself to capturing the subtle intricacies present in language.

Convolutional Neural Networks (CNNs), on the other hand, excel at extracting meaningful information from short text segments. These networks adeptly capture n-gram features and patterns that carry substantial semantic value. As demonstrated by Kim (2014) [9], CNNs exhibit remarkable prowess in text classification tasks, surpassing traditional models while effectively dissecting hierarchical features inherent in sentences.

The third component of our amalgamation, the Long Short-Term Memory (LSTM), emerges as a formidable force in modelling sequential dependencies within the textual content. Leveraging its memory and context-preserving capabilities, LSTM shines in tasks like machine translation, language generation, and sentiment analysis. Tang, Qin, and Liu (2015) [22] present a compelling case for LSTM's aptitude in capturing long-range dependencies within text, effectively elevating their performance in language modelling scenarios.

3.9 Metric scores

In this project, we are working with the classification problem, so various metrics and techniques are employed to measure a model's effectiveness. Most of the metrics selected in this project are based on the confusion metric:

- True Positives (TP)
- False Positives (FP)
- False Negatives (FN)
- True Negatives (TN)

The metrics for this project are listed below:

- Precision: $Precision = \frac{TP}{TP+FP}$. Precision provides insight into the accuracy of positive predictions.

- Recall: $Recall = \frac{TP}{TP+FN}$. The recall provides insight when minimizing false negatives is crucial.
- F1-score: $F1score = \frac{TP}{TP+\frac{1}{2}(FP+FN)}$. In this project, which is a multi-classification problem, we use the F1 macro score.
 $F1_{macro} = \frac{F1_{class1}+F1_{class2}+...+F1_{classn}}{n}$ provides insight. The score harmonizes the trade-off between false positives and false negatives. It is especially useful for imbalanced datasets.
- Matthews's correlation coefficient (MCC) score:

$$MCC = \frac{TP*TN-FP*FN}{\sqrt{(TP+FP)(TP+FN)(TN+FP)(TN+FN)}}$$
. MCC is one of the most reliable statistical measuring scores that is based on all values of the confusion matrix[25]. Davide Chicco and Giuseppe Jurman (2020) [5] showed that MCC produces a more informative score when evaluating binary classifications than the F1 score.

In the context of multi-class models, these evaluation metrics are applied to their macro-averaged variants. Precision, Recall, and the F1-score emerge as stringent and dependable yardsticks, well-established within the realm of classification model assessment. Researchers often rely on these metrics due to their robustness and ability to provide comprehensive insights.

The F1 macro score emerges as a particularly valuable metric when dealing with class imbalances. This measure offers equitable consideration to each class, irrespective of their representation within the data set. However, it's important to note that the F1 macro score can be disproportionately influenced by the majority class's performance in instances of substantial data skew.

Matthews's Correlation Coefficient (MCC), a valuable comparator to the F1 score in situations characterized by severe imbalances. MCC serves as a reliable tool for model selection during the tuning phase. [11] Notably, MCC has gained recognition as a trustworthy metric within multiple research avenues concerning Security Vulnerability Assessment [13].

An intriguing facet of MCC is its consideration of True Negatives within the confusion matrix - an attribute that sets it apart from metrics such as Precision, Recall, and the F1 score. This distinctive attribute contributes to its holistic assessment of model performance, reflecting a nuanced understanding of classification outcomes [12]. In light of these considerations, the choice of MCC as a pivotal metric for model evaluation underscores its significance in navigating the complexities of imbalanced data sets and evaluating classification models effectively.

4 Results

4.1 The overall performance of all data augmentation techniques

First, let's look at the average score for all the data augmentation techniques in table 3. Among the DA techniques, the "None" augmentation technique has the highest MCC score in both the validation (0.514874) and test sets (0.416293), indicating its effectiveness in capturing true positive and true negative instances. Several DA techniques, including "ISD", "DAS", "DAD", "DASyn", and "DAI" consistently achieve relatively high MCC scores across both validation and test sets. This suggests that these techniques perform consistently well in capturing true positives and negatives. Some techniques, such as "US" and "DIS," show lower MCC scores in both validation and test sets, implying that these techniques might not be as effective in handling imbalanced classes as others, which is expected because these techniques remove information from the training dataset. While "DAI" and "OS" show good MCC scores, they might perform better in terms of precision and recall, as evidenced by their relatively high precision and recall scores in the validation and test sets. For now, based on the Matthews Correlation Coefficient (MCC) score, when we don't apply any augmentation technique, it appears to be the most effective choice, closely followed by "ISD," "DAS," "DAD", "DASyn", and "DAI". Let's have a closer inspection of the performance of each model.

DA	Recall_val_avg	Prec_val_avg	F1_val_avg	MCC_val_avg	Recall_test_avg	Prec_test_avg	F1_test_avg	MCC_test_avg
DIS	0.616438	0.589140	0.576400	0.456789	0.554654	0.524413	0.499380	0.335307
ISD	0.611102	0.587936	0.573161	0.453129	0.555971	0.526489	0.499931	0.337735
DAS	0.619783	0.597729	0.582557	0.464738	0.561984	0.536383	0.514163	0.353725
DAD	0.625866	0.603092	0.584587	0.471322	0.564174	0.539203	0.516071	0.363251
DASyn	0.614089	0.627835	0.594442	0.486929	0.558375	0.571369	0.542876	0.388823
DAI	0.619802	0.623875	0.593249	0.485497	0.570918	0.578362	0.550347	0.389116
OS	0.629594	0.626030	0.595943	0.495049	0.582032	0.585021	0.552783	0.399858
None	0.593359	0.680685	0.600078	0.514874	0.552325	0.644627	0.556432	0.416293

Table 3: Performance of every data augmentation technique

4.2 The overall performance of all models

Based on table 4, we can see the performance of all models in the project. The models "SVM," "LR," "XGBoost," "lgbm," "CNN," and "LSTM" consistently exhibit the highest MCC scores on both the validation and test sets. These models generally excel in capturing true positive and true negative instances, making them effective in imbalanced classification tasks. While the models "XGBoost," "lgbm," "CNN," and "LSTM"

achieve high MCC scores, they also have relatively high precision, recall, and F1 scores. This suggests that they strike a good balance between identifying positive instances and avoiding false positives. More complex models like "CNN" and "LSTM" tend to have higher MCC scores compared to simpler model like "KNN." This highlights the importance of capturing intricate patterns and dependencies in NLP tasks, contributing to higher predictive performance. "RF," "XGBoost," and "lgbm" exhibit consistently strong performance across various evaluation metrics. These ensemble methods often leverage the strengths of multiple base models, enhancing their overall performance. Overall, models such as "SVM," "LR," "XGBoost," "lgbm," "CNN," and "LSTM" are proven to be good choices for data augmentation techniques, as they consistently demonstrate higher MCC scores, reflecting their ability to effectively handle imbalanced data. Other factors such as interpretability, computational resources, and the specific goals of the NLP task can be considered but due to the limit of this project, we will only consider their performance.

Comparing the MCC scores from both tables (table 5 and table 6),

Model	Recall_val_avg	Prec_val_avg	F1_val_avg	MCC_val_avg	Recall_test_avg	Prec_test_avg	F1_test_avg	MCC_test_avg
KNN	0.615669	0.588270	0.567823	0.435692	0.547225	0.541731	0.496743	0.326841
RF	0.626207	0.620687	0.580500	0.480053	0.569047	0.556618	0.509415	0.361343
MLP	0.594036	0.597969	0.574604	0.444193	0.545061	0.544213	0.529424	0.369103
SVM	0.656197	0.608215	0.595554	0.492877	0.597867	0.554425	0.537625	0.391770
LR	0.655976	0.610330	0.597731	0.492777	0.597882	0.557315	0.541836	0.393793
XGBoost	0.632557	0.654823	0.615122	0.522448	0.584820	0.602388	0.566719	0.417784
lgbm	0.640087	0.663900	0.619397	0.524344	0.589128	0.602506	0.567711	0.422729
CNN	0.624446	0.665153	0.623953	0.551657	0.556654	0.614226	0.554078	0.432219
LSTM	0.634757	0.667621	0.631287	0.561330	0.564120	0.612660	0.557540	0.436435

Table 4: Performance of every models

Model	Recall_val_avg	Prec_val_avg	F1_val_avg	MCC_val_avg	Recall_test_avg	Prec_test_avg	F1_test_avg	MCC_test_avg
KNN	0.573111	0.670752	0.587988	0.470137	0.521647	0.637242	0.524985	0.362126
RF	0.556648	0.706556	0.558044	0.489874	0.514223	0.643903	0.504247	0.397076
MLP	0.606734	0.628824	0.609032	0.506952	0.573080	0.591058	0.575412	0.412807
SVM	0.604920	0.681906	0.613785	0.552310	0.579721	0.658724	0.586173	0.447186
LSTM	0.630331	0.680576	0.638102	0.578674	0.561376	0.631085	0.565391	0.453081
LR	0.624437	0.694527	0.627845	0.555684	0.593914	0.659293	0.599381	0.455356
CNN	0.621945	0.681497	0.631740	0.574685	0.560519	0.636323	0.567440	0.456455
XGBoost	0.615979	0.728277	0.632381	0.563231	0.579395	0.713337	0.595283	0.461809
lgbm	0.629044	0.731511	0.640620	0.568253	0.570839	0.692655	0.588543	0.474846

Table 5: Performance of every models with no data augmentation

it is evident that in both cases, the models without data augmentation tend to have slightly higher MCC scores compared to the models with the average of all data augmentation techniques. This suggests that, for the specific NLP task at hand, the original models (without augmentation) might perform marginally better in terms of capturing true positive and true negative instances. It's evident that "lgbm" and "XGBoost" consistently perform well regardless of the presence or absence of data augmentation techniques. These models demonstrate their robustness in

Model	Recall_val_avg	Prec_val_avg	F1_val_avg	MCC_val_avg	Recall_test_avg	Prec_test_avg	F1_test_avg	MCC_test_avg
KNN	0.621326	0.577307	0.565142	0.431114	0.550624	0.529037	0.492989	0.322151
RF	0.635007	0.609824	0.583341	0.478811	0.575983	0.545576	0.510068	0.356823
MLP	0.592435	0.594078	0.570261	0.436277	0.541527	0.538305	0.523624	0.363590
SVM	0.662451	0.599228	0.593330	0.485629	0.600080	0.541706	0.531705	0.385012
LR	0.660142	0.599210	0.593753	0.484468	0.598406	0.543846	0.534236	0.385662
XGBoost	0.634591	0.645811	0.613004	0.517444	0.585485	0.588775	0.563214	0.412383
lgbm	0.641442	0.655604	0.616793	0.518956	0.591372	0.591444	0.565154	0.416334
CNN	0.624758	0.663110	0.622980	0.548778	0.556171	0.611464	0.552408	0.429190
LSTM	0.635311	0.666002	0.630435	0.559162	0.564464	0.610357	0.556559	0.434354

Table 6: Performance of every model with data augmentation

handling imbalanced NLP tasks. Additionally, "LSTM" maintains its strong performance, showcasing its capability to capture complex patterns. "CNN", and "LSTM" performance is consistent with data augmentation on average, while "XGBoost" and "lgbm" experienced a dip in performance.

4.3 The performance of top models

As a closer inspection of 4 top performing models, it's interesting to note that the best-performing data augmentation techniques vary across different output models. This highlights the model-specific nature of the effectiveness of augmentation techniques. OS appears to consistently yield good results for both XGBoost and lgbm models, emphasizing its efficacy in enhancing model performance. "DAS" (Data Augmentation with Substitute) and "DASyn" (Data Augmentation with Synonyms) consistently show up as top-performing techniques, underlining their importance in improving model outcomes. Across all four models ("CNN," "LSTM," "XGBoost," and "lgbm"), the top data augmentation techniques consistently exhibit higher MCC scores compared to not using any augmentation technique ("None"). This suggests that data augmentation can enhance the models' ability to handle imbalanced data and improve their predictive power especially complex ensemble models. The incremental improvement in MCC scores across different augmentation techniques underscores the value of leveraging augmented data to mitigate the challenges posed by imbalanced NLP tasks.

DA	MCC_test_avg
None	0.456455
DAS	0.455381
DASyn	0.454664

Table 7: Performance of CNN: top 3 data augmentation techniques

DA	MCC_test_avg
DASyn	0.459909
DIS	0.458049
DAS	0.455307

Table 8: Performance of LSTM: top 3 data augmentation techniques

DA	MCC_test_avg
OS	0.474700
None	0.461809
DAI	0.434455

Table 9: Performance of XGBoost: top 3 data augmentation techniques

DA	MCC_test_avg
OS	0.475589
None	0.474846
DASyn	0.439187

Table 10: Performance of lgblm: top 3 data augmentation techniques

4.4 The overall performance of outputs

Output	Recall_val_avg	Prec_val_avg	F1_val_avg	MCC_val_avg	Recall_test_avg	Prec_test_avg	F1_test_avg	MCC_test_avg
cvss2_authentication	0.555492	0.528102	0.489934	0.478493	0.509161	0.507342	0.457011	0.261379
cvss2_base_score	0.595120	0.589261	0.564899	0.411141	0.523733	0.521450	0.486132	0.261949
cvss2_access_vector	0.601174	0.602817	0.554975	0.513293	0.557218	0.555879	0.510222	0.404208
cvss2_access_complexity	0.602486	0.575658	0.534794	0.430281	0.532332	0.546861	0.488344	0.409125
cvss2_confidentiality_impact	0.667556	0.680363	0.661612	0.504347	0.608289	0.596560	0.581474	0.415115
cvss2_availability_impact	0.646441	0.651065	0.635237	0.477074	0.602641	0.593276	0.582107	0.416922
cvss2_integrity_impact	0.673455	0.668203	0.654601	0.496596	0.622279	0.602629	0.590922	0.431347

Table 11: Performance of every outputs

In table 11, "cvss2_authentication" and "cvss2_base_score" have moderate recall and precision but relatively low F1-score and MCC, suggesting room for improvement. "cvss2_confidentiality_impact" and "cvss2_availability_impact" have higher MCC scores, indicating relatively better overall performance. "cvss2_integrity_impact" has the highest recall and moderate precision, leading to a good F1-score and MCC. The imbalanced distribution of instances across classes can impact the model's performance, particularly for the minority classes. This is evident in varying recall and precision scores. This is expected as we already see the table 1. Based on table 12, and 13, we can see based on the MCC score on the test set that data augmentation techniques performance is not as good as when we applied no data augmentation techniques. However in the validation MCC score, cvss2_authentication and cvss2_access_complexity shows slightly better results. This can be that the testing data set which had the latest

Output	Recall_val_avg	Prec_val_avg	F1_val_avg	MCC_val_avg	Recall_test_avg	Prec_test_avg	F1_test_avg	MCC_test_avg
cvss2_base_score	0.564230	0.651912	0.570744	0.434309	0.488932	0.594290	0.481409	0.270574
cvss2_authentication	0.502278	0.603215	0.518782	0.588488	0.567391	0.716927	0.572039	0.334802
cvss2_availability_impact	0.638236	0.662399	0.635039	0.485736	0.596090	0.608113	0.593468	0.438484
cvss2_confidentiality_impact	0.648895	0.698023	0.653312	0.503844	0.591773	0.625893	0.594818	0.445420
cvss2_integrity_impact	0.645758	0.685409	0.644079	0.491476	0.596744	0.624653	0.597190	0.447057
cvss2_access_vector	0.571053	0.743676	0.571429	0.596115	0.508665	0.700714	0.528311	0.490184
cvss2_access_complexity	0.586485	0.725825	0.610605	0.505789	0.516890	0.642461	0.528804	0.496849

Table 12: Performance of every output with none data augmentation

Output	Recall_val_avg	Prec_val_avg	F1_val_avg	MCC_val_avg	Recall_test_avg	Prec_test_avg	F1_test_avg	MCC_test_avg
cvss2_authentication	0.562608	0.518058	0.486076	0.463784	0.501374	0.479316	0.441629	0.251561
cvss2_base_score	0.599089	0.581210	0.564148	0.408164	0.528204	0.512091	0.486739	0.260841
cvss2_access_vector	0.604856	0.585601	0.552964	0.503170	0.563153	0.538177	0.508011	0.393700
cvss2_access_complexity	0.604410	0.557605	0.525680	0.421203	0.534189	0.535368	0.483480	0.398579
cvss2_confidentiality_impact	0.669888	0.678156	0.662650	0.504410	0.610353	0.592894	0.579806	0.411327
cvss2_availability_impact	0.647483	0.649624	0.635262	0.475973	0.603473	0.591390	0.580664	0.414182
cvss2_integrity_impact	0.676821	0.666112	0.655880	0.497218	0.625383	0.599952	0.590160	0.429438

Table 13: Performance of every outputs average of all data augmentation techniques

description can make data augmentation techniques less effective.

DA	MCC_test_avg
None	0.490184
DASyn	0.441027
OS	0.434261

Table 14: Performance of cvss2_access_vector: top 3 data augmentation techniques

DA	MCC_test_avg
None	0.496849
DASyn	0.455054
OS	0.446651

Table 15: Performance of cvss2_access_complexity: top 3 data augmentation techniques

DA	MCC_test_avg
OS	0.340615
None	0.334802
DAI	0.305624

Table 16: Performance of cvss2_authentication: top 3 data augmentation techniques

For all 7 outputs, we can see that only cvss2_authentication and cvss2_base_score had improvement over the result when we applied no

DA	MCC_test_avg
None	0.447057
US	0.446935
OS	0.437339

Table 17: Performance of cvss2_integrity_impact: top 3 data augmentation techniques

DA	MCC_test_avg
None	0.438484
OS	0.426763
US	0.425425

Table 18: Performance of cvss2_availability_impact: top 3 data augmentation techniques

DA	MCC_test_avg
None	0.445420
OS	0.432558
US	0.423440

Table 19: Performance of cvss2_confidentiality_impact: top 3 data augmentation techniques

DA	MCC_test_avg
DAI	0.281679
OS	0.275904
US	0.275447

Table 20: Performance of cvss2_base_score: top 3 data augmentation techniques

data augmentation techniques. These 2 outputs are also one of the most imbalanced outputs in the dataset. This can show that data augmentation does improve the performance of prediction models in an imbalance data set.

5 Conclusion

5.1 Current results

In this project, we successfully built machine learning and deep learning prediction models to evaluate the vulnerabilities in the NVD datasets. The model includes three feature extraction techniques, seven data-set

balancing techniques, six machine-learning models, and three deep-learning models. The average score MCC on the testing suggested that the data without any data augmentation still generally performed the best. However, like in the previous result from part A of this project, ensemble models such as XGBoost, LGBM and LSTM perform better with data augmentation. These techniques have a slight boost in their performance. Looking at the output performance, the data augmentation techniques improve 2 out of 3 most imbalanced outputs. This shows promise when handling imbalanced data sets using data augmentation techniques. Random oversampling using the top frequent words and data augmentation using synonyms are two of the best data augmentation techniques. This shows the context of vulnerability description is an important factor to prioritise. In conclusion, it can be inferred that data augmentation techniques positively impact the prediction models' performance. Complex models, especially deep learning models, and data augmentation techniques tend to improve the MCC values. Therefore, data augmentation techniques are recommended to enhance the accuracy and reliability of imbalanced models' predictions. Ultimately, the choice of data augmentation technique should be influenced by the specific model being used, and the characteristics of the data distribution in the training set. A well-informed selection can significantly improve results in imbalanced classification tasks.

5.2 Future experiments

New feature selection methods such as BERT (Bidirectional Encoder Representations from Transformers) can be implemented. It is a pre-trained natural language processing model developed by Google [16], which retains a word's context by returning different vectors for the same term depending on the sentences. However, it is compute-intensive and can be a challenge when implemented. Data augmentation with more complexity and contextual focus can be explored. Contextual Augmentation involves generating new sentences by replacing certain words with contextual synonyms to ensure the semantic integrity of the augmented data [10]. Style Transfer technique converts sentences from one style to another while preserving their meaning. It can create diverse sentences with different tones or writing styles for the training data set [19]. We can also use a masked language model [6]. The new augmented data is generated by replacing words with their masked counterparts. We also applied the GPT3.5 and GPT4 from OpenAI but it also required prompt engineer to optimized the new augmented data.

Acknowledgements

I extend my heartfelt gratitude to Dr. Triet Huynh Minh Le, Professor Ali Babar, and the esteemed reviewers for their valuable insights, comments, and suggestions that have significantly enriched this project. Their guidance has played an instrumental role in enhancing the quality and depth of the work.

Furthermore, it is worth noting that this research has been made possible through the support of the University of Adelaide's data science project and the contributions of CREST (Centre for Research on Engineering Software Technologies). The collaboration with these institutions has provided valuable resources and a conducive environment for the successful execution of this endeavor.

A Appendices

Model hyper-parameters

```
models = [
    {
        'name': "LogisticRegression",
        'params': {
            'C': [0.01, 0.1, 1, 10, 100]
        }
    },
    {
        'name': "SVM",
        'params': {
            'C': [0.01, 0.1, 1, 10, 100]
        }
    },
    {
        'name': "KNN",
        'params': {
            'n_neighbors': [11, 31, 51],
            'weights': ['uniform', 'distance'],
            'p': [1, 2]
        }
    },
    {
        "name": "RandomForest",
        "params": {
```



```
        'n_estimators': [100, 300, 500],
        'max_leaf_nodes': [100, 200, 300]
    }
},
{
    'name': "XGBoost",
    'params': {
        'n_estimators': [ 100, 300, 500],
        'max_leaves': [ 100, 200, 300]
    }
},
{
    'name': "LGBM",
    'params': {
        'n_estimators': [ 100, 300, 500],
        'num_leaves': [ 100, 200, 300]
    }
},
]
```

References

- [1] Markus Bayer, Marc-André Kaufhold, and Christian Reuter. A survey on data augmentation for text classification. *ACM Computing Surveys*, 55:1 – 39, 2021.
- [2] Candice Bentéjac, Anna Csörgő, and Gonzalo Martínez-Muñoz. A comparative analysis of xgboost. 11 2019.
- [3] L. Breiman. Random forests. *Machine Learning*, 45:5–32, 2001.
- [4] Joana Cabral Costa, Tiago Roxo, João B. F. Sequeiros, Hugo Proenca, and Pedro R. M. INÁCIO. Predicting cvss metric via description interpretation. *IEEE Access*, 10:59125–59134, 2022.
- [5] Chicco D and Jurman G. The advantages of the matthews correlation coefficient over f1 score and accuracy in binary classification evaluation. *BMC Genomics*, 1 2020.
- [6] Siddharth Dalmia, Brian Yan, Vikas Raunak, Florian Metze, and Shinji Watanabe. Searchable hidden intermediates for end-to-end models of decomposable sequence tasks. In *Proceedings of the 2021 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies*, pages 1882–1896, Online, June 2021. Association for Computational Linguistics.
- [7] Eleonora Fontana. Unsupervised nlp task in python with doc2vec, 2020.
- [8] Chetna Khanna. Text pre-processing: Stop words removal using different libraries, 2021.
- [9] Yoon Kim. Convolutional neural networks for sentence classification, 2014.
- [10] Xiaoyu Kou, Yankai Lin, Shaobo Liu, Peng Li, Jie Zhou, and Yan Zhang. Disentangle-based Continual Graph Representation Learning. In *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pages 2961–2972, Online, November 2020. Association for Computational Linguistics.
- [11] Triet Huynh Minh Le and M. Ali Babar. On the use of fine-grained vulnerable code statements for software vulnerability assessment models. In *Proceedings of the 19th International Conference on Mining Software Repositories, MSR '22*, page 621–633, New York, NY, USA, 2022. Association for Computing Machinery.

-
- [12] Triet Huynh Minh Le, Huaming Chen, and Muhammad Ali Babar. A survey on data-driven software vulnerability assessment and prioritization. *CoRR*, abs/2107.08364, 2021.
 - [13] Triet Huynh Minh Le, David Hin, Roland Croft, and Muhammad Ali Babar. Puminer: Mining security posts from developer question and answer websites with PU learning. *CoRR*, abs/2003.03741, 2020.
 - [14] C. X. Ling and C. Li. Data mining for direct marketing: Problems and solutions. *Kdd*, vol., 98:73–79, 1998.
 - [15] Ruibo Liu, Guangxuan Xu, Chenyan Jia, Weicheng Ma, Lili Wang, and Soroush Vosoughi. Data boost: Text data augmentation through reinforcement learning guided conditional generation. *CoRR*, abs/2012.02952, 2020.
 - [16] Britney Muller. Bert 101 state of the art nlp model explained, 2022.
 - [17] Kamal Nigam, Andrew McCallum, Sebastian Thrun, and Tom Mitchell. Text classification from labeled and unlabeled documents using em. *Machine Learning*, 39:103–134, 05 2000.
 - [18] nvd nist. Cves and the nvd process.
 - [19] Shuo Ren, Wenhui Chen, Shujie Liu, Mu Li, Ming Zhou, and Shuai Ma. Triangular architecture for rare language translation. In *Proceedings of the 56th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 56–65, Melbourne, Australia, July 2018. Association for Computational Linguistics.
 - [20] Connor Shorten and Taghi M. Khoshgoftaar. A survey on image data augmentation for deep learning, 2019. <https://doi.org/10.1186/s40537-019-0197-0>.
 - [21] Georgios Spanos and Lefteris Angelis. A multi-target approach to estimate software vulnerability characteristics and severity scores. *Journal of Systems and Software*, 146:152–166, 2018.
 - [22] Duyu Tang, Bing Qin, and Ting Liu. Document modeling with gated recurrent neural network for sentiment classification. In *Proceedings of the 2015 Conference on Empirical Methods in Natural Language Processing*, pages 1422–1432, Lisbon, Portugal, September 2015. Association for Computational Linguistics.

- [23] Yaming Tang, Fei Zhao, Yibiao Yang, Hongmin Lu, Yuming Zhou, and Baowen Xu. Predicting vulnerable components via text mining or software metrics? an effort-aware perspective. pages 27–36, 08 2015.
- [24] Gissel Velarde, Anindya Sudhir, Sanjay Deshmane, Anuj Deshmunkh, Khushboo Sharma, and Vaibhav Joshi. Evaluating xgboost for balanced and imbalanced data application to fraud detection. 03 2023.
- [25] Voxco. Matthews correlation coefficient: Definition, formula and advantages, 2021.
- [26] Xu Yang and Yi Li. Does data sampling improve deep learning-based vulnerability detection? yeas! and nays! *ACM Computing Surveys*.
- [27] Noah Ziems and Shaoen Wu. Security vulnerability detection using deep learning natural language processing. In *IEEE INFOCOM 2021 - IEEE Conference on Computer Communications Workshops (INFOCOM WKSHPS)*, pages 1–6, 2021.